Real DevOps Interview Q&A — Complete Production Troubleshooting Guide

---

🔥 **1. What will you do if Deployment Fails?**

🧠 **Interview Answer**

👉 First, I will not panic. I will check the CI/CD pipeline logs and identify exactly which stage failed.

👉 I will check whether the failure happened during **code checkout, build, testing, Docker image creation, image push, or Kubernetes deployment**.

👉 If the deployment reached Kubernetes, I will check the deployment, pods, events, and application logs.

👉 After identifying the root cause, I will fix the issue and redeploy.

👉 If the new deployment is already affecting production and the issue cannot be fixed immediately, I will rollback to the previous stable version.

📌 **Why?**

👉 Finding the exact failure point is important because blindly restarting or redeploying can make the problem worse.

🔍 **Step 1 — Check Deployment**

kubectl get deployments

kubectl get pods

Example

NAME READY UP-TO-DATE AVAILABLE

payment-api 0/3 3 0

👉 This tells me the deployment is not successfully running the required replicas.

🔍 **Step 2 — Check Rollout Status**

kubectl rollout status deployment/payment-api

If it is stuck, I investigate further.

🔍 **Step 3 — Check Pod Details**

kubectl describe pod <pod-name>

I will check:

- Events
- Image errors
- Resource problems
- Mount failures
- Scheduling problems

🔍 **Step 4 — Check Logs**

kubectl logs <pod-name>

If the container already restarted:

kubectl logs <pod-name> --previous

🍼 **Real Example**

Suppose the pipeline says:

Docker build successful

Docker push successful

Kubernetes deployment failed

Then I know the problem is probably not the Docker build or registry.

Suppose:

kubectl describe pod <pod-name>

shows:

Failed to pull image

repository does not exist

👉 Root cause: incorrect image name or image tag.

🔧 **Fix**

Correct the image:

kubectl set image deployment/payment-api \

payment-api=myregistry/payment-api:v5
Or rebuild and push the correct image:
docker build -t myregistry/payment-api:v5 .
docker push myregistry/payment-api:v5

✅ **Verify**
kubectl rollout status deployment/payment-api
kubectl get pods
Expected:
payment-api-xxxxx 1/1 Running
payment-api-xxxxx 1/1 Running
payment-api-xxxxx 1/1 Running

🎯 **Interview Line**
"I first identify the exact failed stage, then I investigate the deployment status, pod events and logs, fix the root cause, and verify the rollout. If production is impacted, I can rollback to the last stable version."

🚨 **2. What will you do if a Kubernetes Pod is Crashing?**

🧠 **Interview Answer**

👉 First, I will check the pod status.
👉 If the pod is in CrashLoopBackOff, I will check the current logs and previous container logs.
👉 Then I will use kubectl describe pod to understand events and termination reasons.
👉 I will also check configuration, environment variables, secrets, probes and resource limits.
👉 I will identify the root cause instead of simply restarting the pod.

🔍 **Step 1 — Check Pods**
kubectl get pods
Example:
payment-api 0/1 CrashLoopBackOff

🔍 **Step 2 — Check Logs**
kubectl logs payment-api-xxxxx
Previous container:
kubectl logs payment-api-xxxxx --previous

🔍 **Step 3 — Describe Pod**
kubectl describe pod payment-api-xxxxx

🩺 **Real Example**
Logs show:
Database connection failed
Invalid DB_HOST
👉 Root cause is likely an incorrect database configuration.

🔍 **Check Configuration**
kubectl get configmap
kubectl describe configmap payment-config
Check secrets:
kubectl get secrets

🔧 **Fix**
👉 Correct the configuration according to the application's required values.
Then restart the deployment:
kubectl rollout restart deployment/payment-api

✅ **Verify**
kubectl get pods

kubectl logs <pod-name>
Expected:
payment-api-xxxxx 1/1 Running

🎯 **Interview Line**

"I don't simply restart a crashing pod. I check its status, current and previous logs, events, configuration and resource limits to identify the actual root cause."

📈 **3. What will you do if High Traffic Comes Suddenly?**

🧠 **Interview Answer**

👉 First, I will check whether Horizontal Pod Autoscaling is working.

👉 I will monitor:

- CPU
- Memory
- Request rate
- Response time
- Error rate
- Number of running pods

👉 If autoscaling is working, I will verify that new pods are being created.

👉 If the system is under immediate pressure and autoscaling is insufficient, I can temporarily scale the application manually while investigating the underlying issue.

🔍 **Step 1 — Check HPA**

kubectl get hpa
Example:
NAME TARGETS MINPODS MAXPODS REPLICAS
payment-api 85%/70% 5 50 20

👉 HPA has already increased replicas from 5 to 20.

🔍 **Step 2 — Check Pods**

kubectl get pods

🔍 **Step 3 — Check Resource Usage**

kubectl top pods
kubectl top nodes

🧪 **Real Example**

Normal traffic:
Pods = 5
CPU = 40%
Sale starts:
Requests = 10x
CPU = 90%
HPA should increase:
5 pods → 20 pods → 50 pods

🔍 **If HPA Is Not Scaling**

kubectl describe hpa payment-api
Check whether metrics are available.
kubectl top pods

🔧 **Temporary Fix**

If immediate capacity is required:
kubectl scale deployment/payment-api --replicas=20

✅ **Verify**

kubectl get pods

kubectl get hpa
kubectl top pods
Also monitor application latency and error rate.

🎯 **Interview Line**

"I first verify whether autoscaling is working. If the application is under immediate pressure, I can temporarily increase replicas while investigating why autoscaling did not respond correctly."

💣 **4. What will you do if Production is Down?**

🧠 **Interview Answer**

👉 During a production outage, my first priority is **service recovery**.

👉 I will immediately check monitoring dashboards and alerts.

👉 Then I will check recent deployments or infrastructure changes.

👉 I will investigate the issue layer by layer:

DNS
↓
Load Balancer
↓
Compute
↓
Application
↓
Database

👉 If a recent deployment clearly caused the outage, I will rollback to the last stable version.

🔍 **Step 1 — Check Kubernetes**

kubectl get pods -A
kubectl get nodes
kubectl get svc
kubectl get ingress

🔍 **Step 2 — Check Events**

kubectl get events --sort-by=.lastTimestamp

🔍 **Step 3 — Check Recent Deployment**

kubectl rollout history deployment/payment-api

🩺 **Real Example**

👉 A new application version was deployed 10 minutes ago.

After deployment:
5xx errors increased
Pods are restarting
API latency increased

👉 The deployment is a strong suspect.

🔧 **Immediate Recovery**

kubectl rollout undo deployment/payment-api

✅ **Verify**

kubectl rollout status deployment/payment-api
kubectl get pods
Test the application:
curl -I https://example.com
Expected:
HTTP/2 200

🎯 **Interview Line**

"During a production outage, I prioritize service recovery first. I check monitoring, recent changes and logs, and if a recent release caused the issue, I rollback to the last stable version and then perform detailed root-cause analysis."

🔐 **5. What will you do if you See a Security Issue?**

🧠 **Interview Answer**

👉 First, I will determine the severity and scope of the security issue.

👉 I will analyze logs for suspicious activity.

👉 I will check whether any credentials, tokens or secrets are exposed.

👉 Depending on the situation, I may block malicious traffic using WAF or firewall controls.

👉 I will immediately involve the security team according to the incident-response process.

👉 I will preserve relevant evidence and avoid making uncontrolled production changes.

🔍 **Check Application Logs**

kubectl logs <pod-name>

🔍 **Check AWS Identity**

aws sts get-caller-identity

🍼 **Real Example**

Suppose logs show:
Failed login attempt
Failed login attempt
Failed login attempt
...
from the same suspicious source.

👉 This could indicate a brute-force attempt.

🔧 **Possible Fix**

Depending on the architecture:

👉 Apply WAF rate limiting.

👉 Block malicious traffic.

👉 Temporarily disable compromised credentials.

👉 Rotate exposed secrets.

👉 Isolate affected resources if required.

🚨 **Important**

👉 I would **not blindly block IP addresses or change firewall rules** without understanding the traffic and following the incident-response process.

🎯 **Interview Line**

“I first determine the scope and severity, analyze the evidence, contain the threat using appropriate security controls, inform the security team, and then implement a permanent fix.”

⚡ **6. What will you do if CI/CD Pipeline is Slow?**

🧠 **Interview Answer**

👉 First, I will measure how much time each pipeline stage is taking.

👉 I will not optimize randomly.

Typical pipeline:
Checkout
↓
Build
↓
Test
↓
Docker Build
↓
Docker Push
↓
Deploy

👉 I will identify the actual bottleneck.

🔍 **Example**
Checkout → 30 seconds
Build → 10 minutes
Tests → 5 minutes
Docker Push → 1 minute
Deploy → 1 minute
👉 The build and test stages are the bottleneck.

🔧 **Improvements**
I can use:
👉 Dependency caching
👉 Docker layer caching
👉 Parallel testing
👉 Removing unnecessary steps
👉 Smaller Docker images
👉 Reusing build artifacts

🐬 **Docker Investigation**
docker history payment-api:v5

🧪 **Example**
Without caching:
20 minutes
After optimization:
5 minutes
🎯 **Interview Line**
"I would first measure the duration of every pipeline stage, identify the bottleneck, and then optimize it using caching, parallel execution and build optimization."

🛑 **7. What will you do if Database is Slow?**
🧠 **Interview Answer**
👉 First, I will check database health metrics such as CPU, memory, connections, storage and latency.
👉 Then I will analyze slow queries.
👉 I will check whether missing indexes, inefficient queries or excessive connections are causing the issue.
👉 If the workload is read-heavy, I may consider caching or read replicas depending on the architecture.

🔍 **Application Logs**
kubectl logs <backend-pod>

🔍 **RDS Information**
aws rds describe-db-instances \
--db-instance-identifier production-db

🧪 **Real Example**
Application response:
API latency = 8 seconds
Application logs show:
Database query = 7 seconds
👉 Database query is the bottleneck.
Example:
SELECT * FROM orders
WHERE customer_id = '12345';
👉 If this query scans a large table without a suitable index, performance can degrade.

🔧 **Fix**
Depending on the database and workload:
👉 Optimize the query.
👉 Add an appropriate index after testing.
👉 Use Redis for frequently requested data.
👉 Use read replicas for read-heavy workloads.
👉 Scale the database when the workload actually requires it.

✅ **Verify**
Check:
Database CPU
Database connections
Query latency
API latency
Application error rate
🎯 **Interview Line**
“I first determine whether the database is actually the bottleneck by checking metrics and query performance. Then I optimize the query or indexing and consider caching or read replicas where appropriate.”

🔄 **8. How will you do Deployment Without Downtime?**
🧠 **Interview Answer**
👉 I will use a **rolling update strategy**.
👉 New pods will be created gradually while old pods continue serving traffic.
👉 I will configure readiness probes so traffic is sent only to healthy pods.
👉 I will maintain enough replicas during the update.

🔍 **Check Deployment**
kubectl get deployment
Check rollout:
kubectl rollout status deployment/payment-api

⚙️ **Example Configuration**

```
strategy:
 type: RollingUpdate
 rollingUpdate:
  maxUnavailable: 0
  maxSurge: 1
```

👉 maxUnavailable: 0 helps maintain available capacity during the rollout.

🚀 **Deploy**
kubectl apply -f deployment.yaml

🔍 **Monitor**
kubectl rollout status deployment/payment-api
kubectl get pods -w

🔙 **Rollback if Required**
kubectl rollout undo deployment/payment-api
🎯 **Interview Line**
“For zero-downtime deployment, I use rolling updates with proper readiness probes and enough healthy replicas to continue serving traffic while the new version is deployed.”

🧠 **9. What will you do if Cloud Cost is High?**
🧠 **Interview Answer**
👉 First, I will identify which AWS service or resource is responsible for the increased cost.
👉 I will check billing and usage reports.

👉 Then I will identify unused or oversized resources.
👉 I will review autoscaling configuration and right-size infrastructure.
👉 I will avoid shutting down resources blindly because some may be production-critical.

🔍 **AWS Cost Check**
aws ce get-cost-and-usage \
--time-period Start=2026-08-01,End=2026-08-24 \
--granularity DAILY \
--metrics UnblendedCost

🔍 **Common Causes**
EC2 over-provisioning
RDS oversized instances
Unused EBS volumes
NAT Gateway traffic
S3 storage growth
Unexpected autoscaling
Data transfer

🔧 **Example**
Suppose:
EC2 monthly cost increased by 40%
Investigation shows:
20 instances running
Only 8 are actually required
👉 Right-sizing and removing unnecessary capacity can reduce cost.

✅ **Verify**
👉 Check AWS Cost Explorer/billing again.
👉 Monitor resource utilization.
👉 Confirm production performance has not been affected.
🎯 **Interview Line**
“I first identify the actual cost driver, then optimize resource utilization, right-size infrastructure and review autoscaling instead of blindly shutting down resources.”

🔧 **10. How will you Debug Without Logs?**
🧠 **Interview Answer**
👉 Without logs, troubleshooting becomes much more difficult.
👉 First, I will restore observability.
👉 I will check whether application logs are enabled and whether container stdout/stderr is being collected.
👉 I will use metrics, traces and centralized logging to understand the system behavior.

🔍 **Kubernetes Check**
kubectl logs <pod-name>
If logs are missing:
kubectl describe pod <pod-name>

🔍 **Check Pod Status**
kubectl get pods

🩺 **Example**
Application is failing, but:
kubectl logs <pod-name>
returns no useful information.
👉 I will check whether the application is actually writing logs.
👉 I will also check:

Prometheus metrics
CloudWatch metrics
Application monitoring
Load balancer metrics
Kubernetes events

🔧 **Fix**

👉 Enable application logging.

👉 Send container logs to a centralized logging platform such as:

CloudWatch
ELK
OpenSearch

👉 Add appropriate metrics and distributed tracing if required.

✅ **Verify**

After enabling logging:
kubectl logs <pod-name>
should provide useful application information.

🎯 **Interview Line**

"If logs are missing, I first restore observability instead of making assumptions. I enable centralized logging and use metrics and traces to investigate the problem."

🔥 **FINAL INTERVIEW THINKING**

When the interviewer gives you **any production problem**, don't immediately start listing commands.
Use this sequence:

1. Understand the problem
   ↓
2. Understand the business impact
   ↓
3. Check monitoring and metrics
   ↓
4. Check logs
   ↓
5. Check system status
   ↓
6. Check recent changes
   ↓
7. Identify root cause
   ↓
8. Apply the safest fix
   ↓
9. Verify recovery
   ↓
10. Monitor
   ↓
11. Prevent the issue from happening again

🧠 **FINAL MASTER ANSWER FOR DEVOPS INTERVIEWS**

If the interviewer asks:
"How do you approach production troubleshooting?"
You can answer:
"First, I understand the issue and its business impact. Then I check monitoring metrics, logs, system status and recent changes to identify where the problem started. I troubleshoot layer by layer and isolate the root cause. Once I identify the cause, I take the safest action to restore the service, such as fixing configuration, scaling resources, restarting a failed workload or rolling back an unstable deployment. After recovery, I verify the application and continue monitoring it. Finally, I perform root-cause analysis and implement a permanent preventive solution."

🔥 **ONE-LINE MEMORY FORMULA**

UNDERSTAND
↓
CHECK
↓
INVESTIGATE
↓
FIND ROOT CAUSE
↓
FIX
↓
VERIFY
↓
MONITOR
↓
PREVENT

🧠 **Final Understanding**

👉 **DevOps interviews are not only about knowing Kubernetes, AWS, Docker or Jenkins commands.**

👉 The interviewer wants to know whether you can **think logically during a real production problem**.

👉 A strong DevOps answer should show:

Problem → Investigation → Commands → Evidence → Root Cause → Fix → Verification → Prevention

That is the **real production troubleshooting mindset**.